



Proyecto final

Machine Learning II

Grado en Matemáticas

Autor

Emilio Domínguez y Álvaro Inclán

Madrid, 04/05/2026

Índice

1. Introducción	3
1.1. Contextualización	3
1.1.1. Cancer de pulmón	3
1.1.2. Cancer de colon	4
1.2. Base de datos	5
2. Objetivos del trabajo	5
2.1. Adaptaciones por limitaciones computacionales	6
3. Metodología	7
3.1. Pipeline del trabajo	7
3.2. Preprocesamiento	8
3.2.1. Redimensión	8
3.2.2. LBP features	10
3.3. Modelos implementados	11
3.3.1. CNN (Red Neuronal Convolutacional básica)	11
3.3.2. ResNet (ResNet50)	13
3.3.3. InceptionResNetV2	15
3.4. Modelos no implementados	17
3.4.1. VGG16	17
3.4.2. EfficientNet64	19

3.4.3. MobileNets y Xception	22
3.5. Métricas de evaluación	24
3.6. Estrategia de entrenamiento en Google Colab	25
4. Estructura del código	26
5. Resultados	27
5.1. Resultados del paper original (LBP features, 768×768)	27
5.2. Nuestros resultados (LBP features, 64×64)	27
5.3. Comparación con el paper (diferencia en puntos porcentuales)	28
5.4. Detalle por clase — CNN (único modelo funcional)	28
5.5. Matrices de confusión	28
5.6. Curvas de entrenamiento	29
6. Conclusión	30
6.1. Resumen hallazgos	30
6.2. Análisis del rendimiento de la CNN	30
6.3. Análisis del fracaso de ResNet50 e InceptionResNetV2	31
6.4. ¿Por qué la CNN sí funciona y los modelos preentrenados no?	32
6.5. Lecciones aprendidas	33
6.6. Posibles mejoras	33
6.7. Conclusión final	34

1. Introducción

El cáncer es una de las principales causas de mortalidad a nivel mundial. En particular, el cáncer de pulmón contribuye al 18.4% de las muertes por cáncer a nivel global, mientras que el cáncer de colon constituye el 9.2% (Bray et al., 2018). La detección temprana de estas enfermedades es crucial para mejorar las tasas de supervivencia de los pacientes.

La clasificación automática de imágenes histopatológicas mediante técnicas de aprendizaje profundo (deep learning) representa una herramienta prometedora para asistir a los patólogos en el diagnóstico. Este enfoque permite analizar grandes volúmenes de imágenes de forma rápida y eficiente.

1.1. Contextualización

1.1.1. Cancer de pulmón

El tejido pulmonar benigno se refiere a crecimientos o nódulos no cancerosos en el pulmón, generalmente asintomáticos, pequeños, redondos y de bordes definidos. Más de la mitad de los nódulos encontrados son benignos, causados frecuentemente por infecciones pasadas.

El adenocarcinoma de pulmón es un tipo de cáncer no microcítico de pulmón que se origina en las glándulas que secretan mucosidad. Es el tipo más frecuente de cáncer de pulmón puesto que constituye cerca del 30% de los casos, y es la neoplasia maligna más frecuente de pulmón entre las personas que no fuman y los adultos de menos de 45 años.

El carcinoma de células escamosas de pulmón, o carcinoma epidermoide, es un tipo de cáncer no microcítico de pulmón que suele presentarse en uno de los bronquios pulmonares. Ocupa el segundo lugar en frecuencia entre los cánceres no microcíticos de pulmón y constituye aproximadamente el 30% de todos los casos de cáncer de pulmón. Por lo general, este es un tipo de cáncer de crecimiento lento, pero con el

tiempo puede extenderse a otras partes del cuerpo.

El tratamiento como en cualquier tipo de cancer se basa principalmente en cirugía, quimioterapia y radioterapia. En ocasiones es posible aplicar también terapias dirigidas e inmunoterapia. El tratamiento se administra dependiendo del cuadro clínico de cada paciente.

1.1.2. Cancer de colon

El tejido benigno de colon incluye principalmente los pólipos. Aunque no son cancerosos inicialmente, algunos tipos de pólipos tienen el potencial de transformarse en cáncer si no se detectan y extirpan a tiempo.

El adenocarcinoma de colon es el tipo más común de cáncer de colon. Se origina en las células formadoras de glándulas que recubren la superficie interna del colon. Estas células normalmente producen moco, que facilita el tránsito de las heces por el intestino grueso. Cuando estas células se vuelven anormales y crecen descontroladamente, forman un tumor llamado adenocarcinoma.

Para muchos pacientes, la cirugía para extirpar el segmento de colon que contiene el tumor y los ganglios linfáticos adyacentes es el primer paso. Se puede recomendar quimioterapia después de la cirugía si el estadio o las características del tumor sugieren un mayor riesgo de recurrencia. En algunas situaciones, especialmente en el caso de cánceres de recto o tumores de colon muy voluminosos, se puede administrar quimioterapia o radioterapia antes de la cirugía para reducir el tamaño del tumor. Supongamos que el tumor presenta biomarcadores específicos, como dMMR, amplificación de HER2 o alteraciones específicas de BRAF. En ese caso, su oncólogo podría considerar la inmunoterapia o la terapia dirigida como parte de la atención estándar o de un ensayo clínico.

1.2. Base de datos

El dataset utilizado en este trabajo es el LC25000 (Lung and Colon Cancer Histopathological Image Dataset), creado por Borkowski et al. (2019). Sus características principales son:

- 25,000 imágenes a color en formato JPEG.
- 5 clases con 5,000 imágenes cada una:
 1. colon_aca: Adenocarcinoma de colon
 2. colon_n: Tejido benigno de colon
 3. lung_aca: Adenocarcinoma de pulmón
 4. lung_n: Tejido benigno de pulmón
 5. lung_scc: Carcinoma de células escamosas de pulmón
- Resolución original: 768×768 píxeles.
- Las imágenes originales (1,250) fueron aumentadas mediante rotaciones (hasta 25°) y flips horizontales/verticales para obtener las 25,000 finales.
- El dataset ya viene particionado en dos conjuntos:

Train and Validation Set: Aproximadamente 4,500 imágenes por clase (22,501 imágenes en total).

Test Set: Aproximadamente 500 imágenes por clase (2,499 imágenes en total).

2. Objetivos del trabajo

Replicar parcialmente los resultados del paper de Alsubai (2024), evaluando el impacto de las características LBP en la clasificación de imágenes histopatológicas del dataset LC25000. Concretamente nos centraremos en los siguientes objetivos:

1. Implementar y evaluar tres modelos de los siete propuestos en el paper:
 - CNN (Red Neuronal Convolutacional básica)
 - ResNet (Red Residual - ResNet50)
 - InceptionResNetV2 (modelo híbrido Inception + ResNet)
2. Aplicar preprocesamiento LBP (Local Binary Patterns) a las imágenes, tal como se describe en el paper.
3. Comparar los resultados obtenidos con los del paper original, analizando las diferencias derivadas de nuestras limitaciones computacionales.

2.1. Adaptaciones por limitaciones computacionales

Debido a que ejecutaremos el código en Google Colab (entorno con recursos limitados en tiempo y GPU), realizamos las siguientes adaptaciones respecto al paper original:

Cuadro 1: Comparativa entre el paper original y nuestro trabajo

Aspecto	Paper original	Nuestro trabajo
Resolución de imágenes	768×768 px	64×64 px
Modelos evaluados	7 (CNN, VGG16, ResNet, EfficientNetB4, MobileNet, Xception, InceptionResNetV2)	3 (CNN, ResNet, Inception-ResNetV2)
Entorno de ejecución	Dell PowerEdge T430 GPU	Google Colab (GPU compartida)
Preprocesamiento	Full features + LBP features	LBP features
Partición de datos	70:30 (train:test)	Usamos la partición proporcionada ($\sim 90:10$) con split interno de validación (80:20 dentro del train)

La reducción de resolución de 768×768 a 64×64 implica una pérdida significativa de información visual, lo que probablemente resultará en métricas inferiores a las del paper. Sin embargo, esto nos permite entrenar los modelos dentro de las restricciones de Google Colab.

3. Metodología

3.1. Pipeline del trabajo

El pipeline completo del trabajo sigue estos pasos:

1. Imágenes LC25000 (768×768)
2. Redimensionado (64×64)

3. Extracción LBP Features y normalización [0,1]
4. Entrenamiento de modelos (CNN, ResNet, InceptionResNetV2)
5. Evaluación y comparación (Accuracy, Precision, Recall, F1-Score)

3.2. Preprocesamiento

3.2.1. Redimensión

La redimensión de imágenes es un paso esencial en el preprocesamiento de datos para Deep Learning. Básicamente, consiste en ajustar todas las imágenes de tu conjunto de datos a un tamaño uniforme (ancho y alto específicos) antes de pasarlas por la red neuronal.

La mayoría de las arquitecturas de redes neuronales (como las CNN) tienen una capa de entrada con dimensiones fijas. Si las imágenes varían de tamaño, el número de píxeles cambiaría y la arquitectura tendría que modificarse (lo cual no tiene sentido).

Hay diferentes métodos:

- Interpolación (Resizing): Se recalculan los píxeles para ajustar la imagen al nuevo tamaño.
- Recorte (Cropping): Si no quieres deformar la imagen, puedes tomar solo el centro o una parte aleatoria que cumpla con el tamaño requerido.
- Relleno (Padding): Si tienes imágenes muy alargadas, puedes añadir bordes negros (o de otro color) para convertirlas en un cuadrado sin alterar la proporción original del objeto.

Nosotros en nuestro trabajo utilizamos el método de interpolación. Concretamente utilizamos dos tipos: interpolación Lanczos para la primera parte del trabajo e interpolación bilineal para entrenar el modelo InceptionResNetV2 (explicaremos en detalle más adelante el por qué de esta decisión cuando tratemos el modelo).

La interpolación es el proceso matemático de estimar valores desconocidos en posiciones intermedias a partir de valores conocidos (los píxeles originales). Cuando redimensionas una imagen, estás creando una nueva cuadrícula de píxeles. Como esa nueva cuadrícula casi nunca coincide perfectamente con la original, la interpolación decide qué color poner en esos nuevos espacios.

En la interpolación bilineal el nuevo pixel es el promedio ponderado de los 4 píxeles originales más cercanos (en un cuadrado de 2×2). Por otra parte, la interpolación de Lanczos es una de las técnicas de redimensión más avanzadas y precisas en el procesamiento de imágenes digitales y visión por computadora. Utiliza la función matemática sinc ventaneada para calcular el valor del nuevo pixel.

- Función Sinc: Utiliza la función $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$ para reconstruir la señal digital. Esta función es infinita y no se puede usar directamente en ordenadores.
- Ventana de Lanczos: El algoritmo trunca la función usando una ventana de tamaño fija definida por un parámetro a . Por ejemplo, Lanczos-2 analiza un área de 4×4 píxeles ($2a \times 2a$, es decir $a = 2$).

Por tanto, el proceso para calcular el valor del pixel es:

1. Identificamos el vecindario en función del parámetro a .
2. Calculamos distancias: Para cada uno de esos píxeles, se mide la distancia horizontal (dx) y vertical (dy) respecto al nuevo punto.
3. Obtenemos los pesos: Se aplica la fórmula $L(x)$ a esas distancias. Obtienes un peso para la x y otro para la y . El peso final de ese píxel vecino es el producto de ambos: $W = L(dx) \cdot L(dy)$. Donde $L(x)$ es:

$$L(x) = \begin{cases} \text{sinc}(x) \cdot \text{sinc}\left(\frac{x}{a}\right) & \text{si } -a < x < a \\ 0 & \text{en otro caso} \end{cases}$$

4. Ponderación: Se multiplica el color de cada píxel original por su peso W .

5. Normalización: Divides entre la suma de todas las ponderaciones.

En nuestro trabajo consideramos la reducción a un tamaño estandar de 64x64, debido a nuestra capacidad de computo limitada, intentando no perder una precisión razonable para este tipo de problemas.

3.2.2. LBP features

LBP (Local Binary Pattern) es un descriptor de textura ampliamente utilizado en visión por computador. Es un método manual que resume la estructura local de una imagen. Su funcionamiento es el siguiente:

1. Vecindario: Para cada píxel de la imagen, se examina su vecindario (un conjunto de píxeles circundantes).
2. Umbralización (Thresholding): Cada píxel vecino se compara con el píxel central, se compara el valor de intensidad:
 - Si el valor del vecino es mayor o igual al del centro entonces se le asigna un 1.
 - Si es menor entonces se le asigna un 0.
3. Generación del código binario: Se genera un patrón binario que se convierte a un número decimal, representando la información de textura local. Se leen estos ceros y unos en orden (normalmente siguiendo las agujas del reloj) para formar un número binario de 8 bits. Ese número binario se convierte a decimal. Este nuevo valor sustituye al píxel original.
4. Se construye un histograma de estos valores LBP, proporcionando una representación compacta de la textura. No se le pasa la imagen resultante a la red directamente. El histograma cuenta cuántas veces aparece cada valor y este histograma es el vector de características (feature vector). Es una firma digital que describe si la imagen tiene muchos bordes, manchas, esquinas o zonas planas. La red trabaja con el vector de características.

El LBP se solía usar antes del auge del Deep Learning como entrada para clasificadores como SVM o Random Forest, aunque hoy en día a veces se usa como una capa de preprocesamiento para ayudar a la red a enfocarse en la textura.

En nuestro caso, aplicamos LBP sobre cada canal de la imagen RGB por separado, generando una imagen LBP de 3 canales que captura la información de textura de la imagen original. Esta transformación tiene dos ventajas:

- Robustez ante cambios de iluminación: LBP es invariante a transformaciones monótonas de la escala de grises.
- Eficiencia computacional: la representación LBP puede mejorar la capacidad discriminativa de los modelos al destacar patrones de textura relevantes.

3.3. Modelos implementados

Como hemos comentado anteriormente aplicamos 3 de los 7 modelos del paper original. Vamos a explicar ahora cada modelo detenidamente, así como su estructura.

3.3.1. CNN (Red Neuronal Convolutiva básica)

Es un tipo de red neuronal artificial diseñada específicamente para procesar y analizar datos visuales, como imágenes o vídeos. Como toda estructura de inteligencia artificial, su funcionamiento se inspira en el cuerpo humano (en este caso en la corteza visual humana), donde diferentes grupos de neuronas se especializan en detectar características específicas. En lugar de mirar una imagen como una lista plana de números, la CNN entiende la disposición espacial de los píxeles para reconocer formas y patrones. Por tanto, una CNN no es una sola capa, sino una secuencia de diferentes tipos de capas que transforman la imagen paso a paso. Sus componentes principales son:

1. Capa Convolutiva (Convolutional Layer): Es la parte central de la red. Utiliza

pequeños filtros llamados kernels que se deslizan sobre la imagen para detectar características básicas como bordes, colores o texturas.

2. Capa de Activación (ReLU): Normalmente se aplica después de la convolución. Su función es introducir no linealidad en el modelo, permitiendo que la red aprenda patrones complejos y decida qué información es relevante.
3. Capa de Agrupación (Pooling): Sirve para reducir el tamaño de los datos (submuestreo) manteniendo la información más importante. Esto hace que la red sea más rápida y eficiente al procesar menos píxeles.
4. Capa Completamente Conectada (Fully Connected Layer): Se encuentra al final de la red. Toma todas las características detectadas por las capas anteriores y las combina para tomar la decisión final.

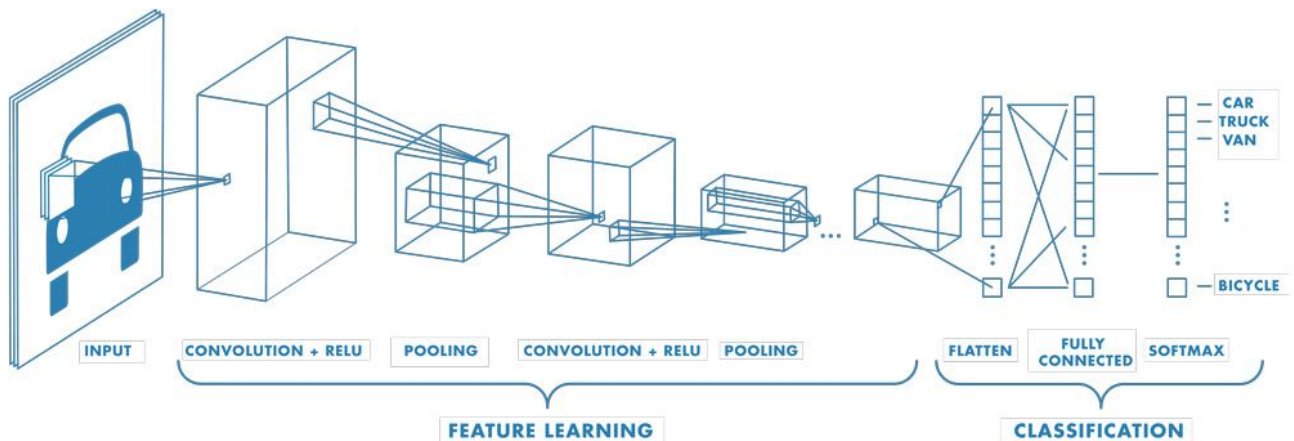


Figura 1: Estructura de un modelo CNN convencional

En nuestro caso, implementamos una CNN desde cero con la siguiente arquitectura:

- 3 bloques convolucionales con filtros progresivamente más profundos (32, 64, 128).
- Cada bloque incluye: Conv2D (3×3, padding='same') - BatchNormalization - ReLU - MaxPooling2D (2×2).

- Capas densas: Dense(256) - BatchNorm - Dropout(0.5) - Dense(128) - BatchNorm - Dropout(0.3).
- Capa de salida Dense(5, softmax) para las 5 clases.
- Optimizador: Adam (lr=0.001). Loss: categorical crossentropy.

3.3.2. ResNet (ResNet50)

Es un tipo avanzado de CNN introducida por Microsoft en 2015 ("Deep Residual Learning for Image Recognition" por Kaiming He, Xiangyu Zhang, Shaoqing Ren y Jian Sun, publicado en 2015 por Microsoft Research.). Su gran avance fue permitir el entrenamiento de redes extremadamente profundas sin que su rendimiento empeorara.

Para ello, el problema que resuelve es el problema del gradiente desvanecido. Básicamente consiste en que los gradientes que indican a la red cómo mejorar se vuelven tan pequeños que son ínfimos. De esta forma, llega un punto donde la red no consigue seguir aprendiendo.

La innovación que introduce es el bloque residual. En lugar de que cada capa aprenda una transformación completa desde cero, ResNet utiliza una conexión de salto o residual. La red toma la información de entrada y la pasa directamente a una capa posterior, saltándose algunas capas intermedias. Al final del bloque, la información que 'saltó' se suma a la salida de las capas que sí trabajaron.

Componentes de un bloque residual típico:

- Capas Convolucionales: Generalmente dos o tres capas que extraen características.
- Batch Normalization (BN): Estabiliza el entrenamiento normalizando las salidas de las neuronas.
- Activación ReLU: Introduce no linealidad para aprender patrones complejos.

- Atajo (Shortcut): El camino directo que conecta la entrada con la salida del bloque.
- Adición elemento a elemento: La operación matemática que combina la identidad original con el residuo aprendido.

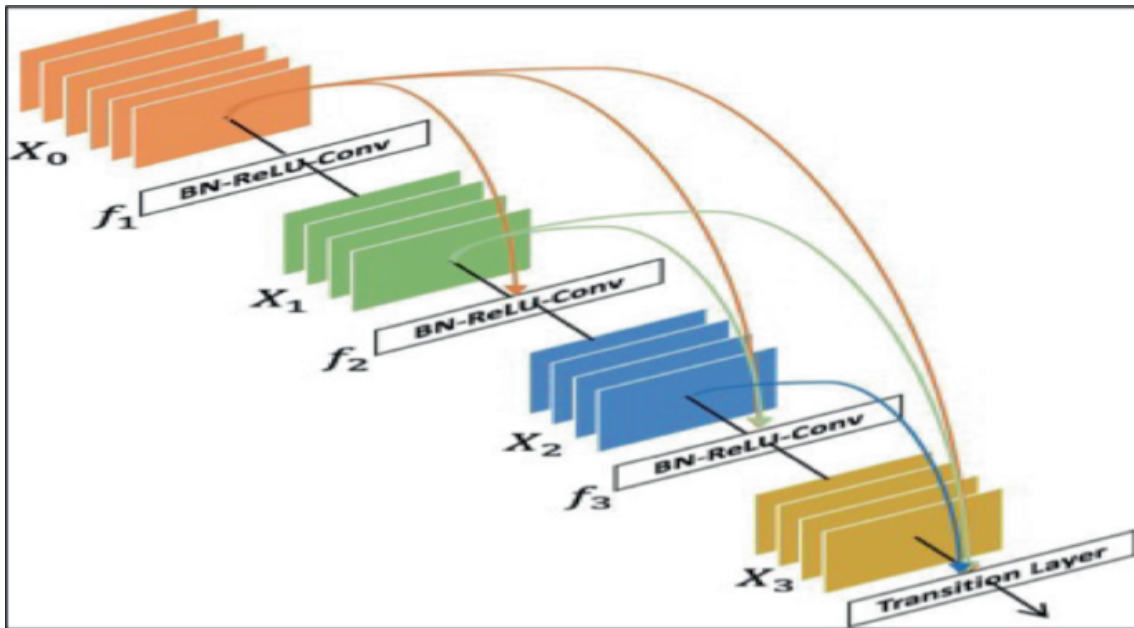


Figura 2: Estructura de un modelo ResNet

Utilizamos ResNet50 preentrenada en ImageNet con ‘include_top=False’ y ‘pooling=‘max’’, con todas las capas entrenables (fine-tuning completo). La cabeza de clasificación consiste en BatchNormalization - Dense(256, regularización L2+L1) - Dropout(0.45) - Dense(5, softmax). Se utilizó el optimizador Adamax (lr=0.001).

Preentrenar una ResNet con ImageNet consiste en entrenar la red neuronal utilizando una de las bases de datos más grandes y famosas del mundo para que aprenda a reconocer objetos antes de usarla para una tarea específica. Este proceso es la base del Aprendizaje por Transferencia (Transfer Learning), donde el modelo aprovecha el conocimiento previo para aprender nuevas tareas mucho más rápido y de forma más óptima. Además, necesita menos datos para lograr una alta precisión.

3.3.3. InceptionResNetV2

Es una de las redes más potentes y complejas en visión por computadora, diseñada para extraer la máxima información posible de una imagen con un coste computacional optimizado. Presentada en el paper Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. A. (2017). "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning". Proceedings of the AAAI Conference on Artificial Intelligence.

Es una arquitectura de red neuronal híbrida que combina la eficiencia de los módulos Inception (de Google), para una extracción eficiente de características a múltiples escala, y la estabilidad de las Conexiones Residuales (ResNet), para facilitar el entrenamiento profundo.

La combinación consiste en:

- Inception (ancho): En lugar de elegir un único filtro en una capa, el bloque Inception aplica varios tamaños de filtro a la vez en paralelo. Esto permite que la red detecte patrones de distintos tamaños simultáneamente.
- ResNet (profundidad): Añade las conexiones de salto (shortcuts). La salida de los bloques Inception se suma a la entrada original. Esto permite que la red sea muchísimo más profunda sin que el gradiente se pierda.

Componentes principales de la red:

- Bloques Residuales-Inception: Son módulos donde el procesamiento paralelo de filtros se combina mediante una suma residual al final del bloque.
- Reducción de tamaño (Reduction Blocks): Utiliza bloques especiales para reducir el ancho y alto de la imagen (pooling) de forma eficiente, evitando cuellos de botella de información.
- Escalado Residual: Debido a que la red es muy profunda, los valores de las capas pueden volverse muy grandes. Esta arquitectura escala las activaciones residuales antes de sumarlas, lo que estabiliza el entrenamiento.

- Stem mejorado: La parte inicial de la red es más sofisticada que en versiones anteriores, preparando mejor los datos antes de entrar a los bloques principales.

Inception Resnet V2 Network

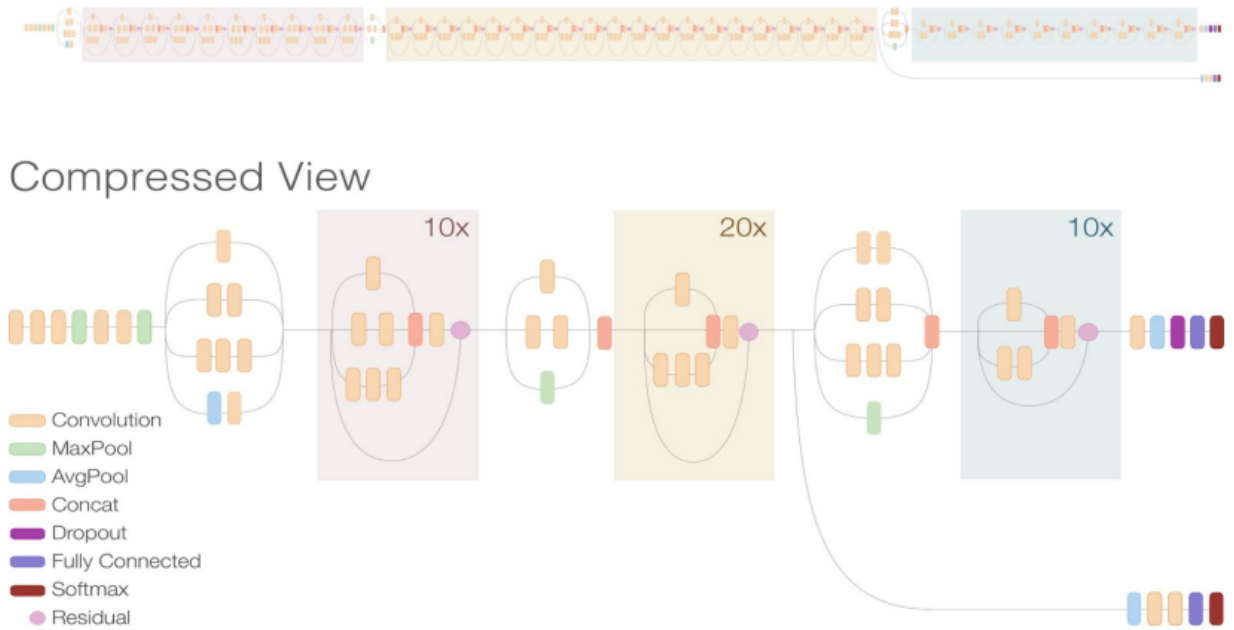


Figura 3: Estructura de un modelo InceptionResNetV2. Imagen sacada de David Solanas Sanz - Trabajo Fin de Grado "Predicción del diagnóstico de la enfermedad de Alzheimer mediante deep-learning en imágenes 18F-FDG PET". Universidad de Zaragoza.

Utilizamos el modelo preentrenado en ImageNet con `'include_top=False'` y `'pooling='max'`, con todas las capas entrenables. Dado que InceptionResNetV2 requiere un input mínimo de 75×75 píxeles, se añadió una capa de redimensionado (`'tf.image.resize'`) que escala las imágenes de 64×64 a 75×75 mediante interpolación bilineal antes de alimentar el modelo base. La cabeza de clasificación es idéntica a la de ResNet50 (BatchNormalization - Dense(256, L2+L1) - Dropout(0.45) - Dense(5, softmax)), con optimizador Adamax ($lr=0.001$).

3.4. Modelos no implementados

En el paper original Alsubai (2024) se hace una comparacion de 7 modelos distintos. Cada uno de estos modelos se entrena dos veces. Con la base de datos original y con la base de datos tras la realización de LBP features.

Como ya hemos comentado previamente debido a nuestras limitaciones tecinas hemos decidido centrarnos en los modelos que considerabamos mas interesantes o los que esperabamos obtener mejor resultado. Dentro del paper se han analizado distitas arquitecturas que aunque no hemos corroborado entrenado las respectivos modelos hemos considerado interesante comentar las distitnas estructuras en el trabajo.

3.4.1. VGG16

Vamos a ver la estructura completa que tiene la arquitectura VGC que fue presentada en la conferencia ICLR en 2015(VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION). La estructura original fue propuesta por Simonyan y Zisserman. El 16 viene del numero de capas que tiene esta estructura (13 capas son de convolución y 3 capas Fully conected)

- Lo primero que recibe el ordenador es una imagen de tamaño 224x224x3 y hacemos pasar la imagen por dos capas de convolución. Cada una de estas capas le hacen pasar por 64 mapas de caracteristica de tamaño 3x3 (con paso 1). Esto nos genera un mapa de caracteristicas de tamaño 224x224x64.
- Ahora tomamos nuestro mapa de caracteristica y realizamos un max polling de tamaño 2x2 con paso 2. Esto reduce nuestro mapa de caracteristica a un tamaño de 112x112x64
- Ahora se realizan dos capas de convolucion con 128 caracteristicas. Cada caracteristica tiene la mimsa estructura que las primeras capas de convolución (tamaño 3x3 con paso 1). Esto nos deja una estructura con 112x112x128 datos.

- Realizamos una capa de Max pooling con las mismas características que previamente (tamaño 2x2 con paso 2). Esto nos deja un tamaño de 56x56x128
- Ahora se realizan 3 capas de convolución consecutivas. En cada una de ellas toma 256 características distintas de tamaño 3x3 con paso 1. Esto nos da un mapa de características de 56x56x256.
- Se realiza otra capa de maxpooling con las mismas características que previamente. Con tamaño 2x2 y paso 2. Esto reduce nuestro mapa de características a tamaño 28x28x256.
- Ahora se realizan 3 capas de convolución con 512 características. Los mapas de características se obtienen con la misma estructura que antes (tamaño 3x3 con paso 1). Esto nos deja un mapa de características de 28x28x512.
- Ahora se realiza una capa de Maxpooling con las mismas características que antes (mapa de 2x2 con paso 2) dejándonos con un mapa de características de 14x14x512.
- Ahora realizamos 3 capas de convolución donde aplicamos 512 filtros (o características) con tamaño 3x3 y paso 1. Esto nos deja un mapa de características final de 14x14x512
- Se realiza una última capa de MaxPooling de tamaño 2x2 con paso 2 para obtener un último mapa de característica de tamaño 7x7x512.
- Ahora se aplanan los mapas de características para que sirvan como entrada para una capa de entrada para una red neuronal con 25088 parámetros.
- Ahora tomamos dos capas que sirvan como las capas ocultas de la red neuronal. Cada una tendrá 4096 perceptrones. Para evitar problemas como el overfitting o el problema del desvanecimiento del gradiente (vanishing gradient) se realiza dropout y todas las funciones de activaciones son la función ReLU o alguna de sus variantes.

- Debido a que en el paper se estaba entrenando una base de datos ImageNet ,para la competición ImageNet Challenge 2014, tiene 1000 clases distintas por lo que esta última capa de output (o softmax) se adaptaría en nuestro trabajo a solo 5 si la ajustáramos.

Podemos ver que es una arquitectura bastante simple donde lo que se está realizando es intercalar capas de convolución con capas de max pooling para terminar con una red neuronal sencilla con dos capas ocultas.

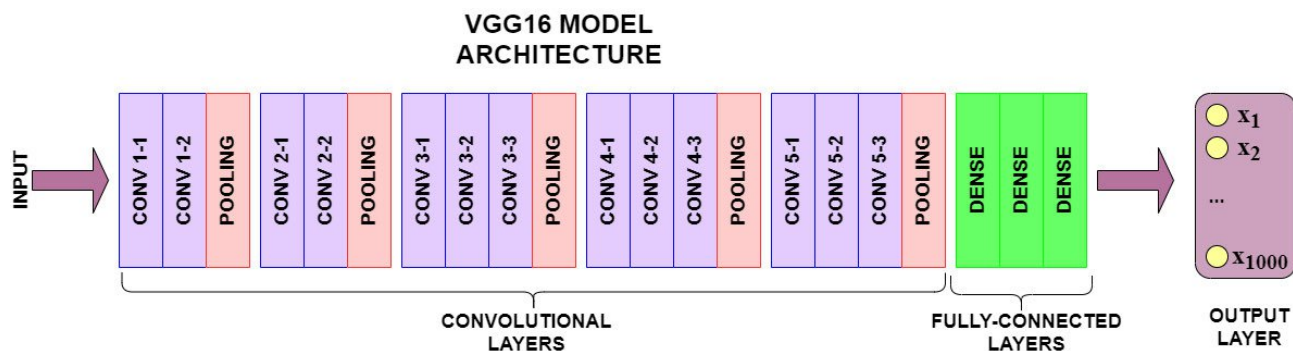


Figura 4: Estructura de los modelos VGG16

En el momento en el que se presentó esta arquitectura llamó mucho la atención pero se ha visto que tiene grandes fallos que han evitado su implementación en la industria. Esta arquitectura tiene alrededor de 138 millones de parámetros que tiene que optimizar nuestro modelo. Esto genera un coste computacional grande a la hora del entrenamiento (en el tiempo y en el espacio que ocupa). Aun con este defecto es una arquitectura que se considera como el fundamento de muchas arquitecturas distintas y se sigue analizando a un nivel académico o teórico.

3.4.2. EfficientNet64

Vamos a hablar de un tipo de arquitectura que se desarrolló en el departamento de IA de Google (Google AI). Este tipo de arquitecturas tiene como objetivo escalar las CNN a tamaños mayores de una manera eficiente. Este tipo de métodos nos ayuda

a encontrar un algoritmo o un par de reglas para poder escalar arquitecturas a otras mas complejas (como por ejemplo podriamos escalar una VGG16 a una con 3 capas mas consiguiendo una arquitectura VGG19).

Para poder hablar de escalar CNN tenemos que dejar claro que significa escalar. Siempre que hablamos de escalar estamos hablando escalar a una mayor escala.

- **Width scaling** Es un tipo de escala en la que no tocamos el numero de capas y solo nos centramos en el tamaño de cada unas de estas capas. En el caso de Width scaling estamos aumentando el numero de filtros o características en cada una de las capas de convolución. Aqui estamos consideramos que aumentamos lo ancho (width) de nuestra CNN

- **Depth scaling** En este tipo de escala estamos aumentando el numero de capas que tendra nuestra CNN. Aqui consideramos que aumentamos la profundidad (depth) de nuestra red CNN.

- **Resolution scaling** En este tipo de escala estamos evitando reducir la resolución de la imagen original a 224x224 (como haciamos en el VGG16 por ejemplo) y la redimensionamos a un tamaño de 360x360.

- **Compound scaling** En este tipo de escala hacemos una combinación de las tres anteriores.

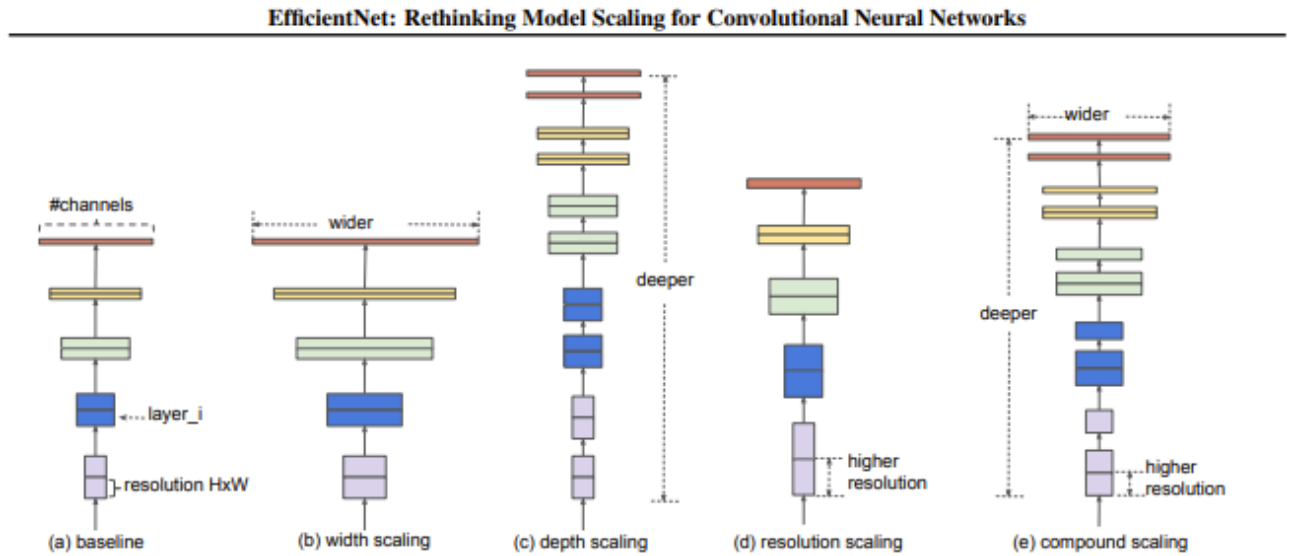


Figura 5: Distintos tipos de escalar una red CNN. Tenemos la red original (a) y vemos los modelos que se centran en aumentar el "ancho", lo "profundo", o aumentar la escala del input (modelos (b-d)) y la última escala que hace todos a la vez.

En el paper que se habla de escalar redes neuronales preexistentes se tienen en cuenta de que los factores de depth, width y resolution eran valores que no se podían tocar individualmente para los mejores resultados. Es una idea intuitiva, si tenemos una red neuronal con más capas tendrá más sentido aumentar de alguna manera la resolución original para tener más información que se pueda usar.

La principal idea que se muestra en el paper (y es la base de los modelos EfficientNet) es encontrar un equilibrio entre aumentar width, depth y resolution. Vamos a ver rápidamente como se describe sin entrar mucho en el tema.

$$\text{depth: } d = \alpha^\phi$$

$$\text{width: } w = \beta^\phi$$

$$\text{resolution: } r = \gamma^\phi$$

$$\text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Donde el valor de ϕ Viene dando en funcion del coste computacional que aporta cada uno de estos parametros a el modelo final. La expresion que he puesto previamente representaria un aumento de 2^n de recursos a la hora de la computacional. Este coste computacional que hemos comentado es el numero de operaciones que se realizan por un segundo determinado por FLOPS. Es un valor que se le puede añadir a la funcion de coste que estamos analizando en nuestro modelo para mejorar la inferencia de nuestro modelo manteniendo una relación con datos precisos. Ellos obtienen la siguiente expresión.

$$ACC(m) \times [FLOPS(m)/T]^w$$

En esta exprecion se tienen en cuenta u valor de Y que se refiere a los flops en el caso previo del escalado, un valor w que es un hiperparametro que se puede ajustar pero en el paper obtienen por un metodo de grid search como 0.07.

El metodo EfficientNet aunque por su nombre se determine que sea eficiente a la hora del reescalado de los modelos CNN cuando trabajemos con un presupuesto no hemos querido implementarlo porque no pensabamos que podriamos determinar cuando podemos parar en reescalar el modelo. Es un metodo útil en la empresa pero en nuestro caso que no nos hemos enfrentado a un problema de ajustar presupuesto lo desechamos la implementación y nos centramos en los que ya hemos comentado.

3.4.3. MobileNets y Xception

Vamos a ver los dos últimos modelos que se basan en la misma idea. Depthwise Separable convolution. Este concepto nace cuando los matematicos se dieron cuenta de lo costoso que es para nuestro ordenador este tipo de modelos (a la hora de inferir datos y entrenar los modelos). Este tipo de arquitectura busca optimizar el coste del modelo sacrificando la minima precisión del modelo.

Vamos a ver como se comportan los Depthwise Separable convolutions. Estas arquitecturas tienen dos pasos bien definidos.

- **Depthwise Convolution: Filtering Stage ("Fase de filtrado")** Vamos a poner un ejemplo para poder explicarlo lo mas claramente posible. Tenemos un mapa de características de tamaño $D_f \times D_f \times M$. Puede ser el input original o el paso tras una convolución previa. En este caso vamos a aplicar filtros (buscar características) con tamaño $D_k \times D_k \times 1$. Este filtro solo lo aplicaremos a una característica de la que hemos obtenido previamente (a un valor de la 3 componente). Esto nos hace tener obligatoriamente M filtros. Cuando aplicamos esta convolución (estos filtros o estas características) obtenemos un output de tamaño $D_g \times D_g \times M$

- **Pointwise Convolution: Combination Stage ("Fase de combinación")** Ahora tomamos el output final que hemos obtenido y vamos a realizar a cada cuadrícula del espacio $D_g \times D_g$ y en esa cuadrícula vamos uno por uno tomando elementos de tamaño $1 \times 1 \times M$ y realizaremos una combinación lineal de estos valores. Esta combinación lineal será uno de los elementos mas importantes para determinar un filtro. Podremos considerar N combinaciones lineales distintas y obtendremos un output de tamaño $D_G \times D_g \times N$.

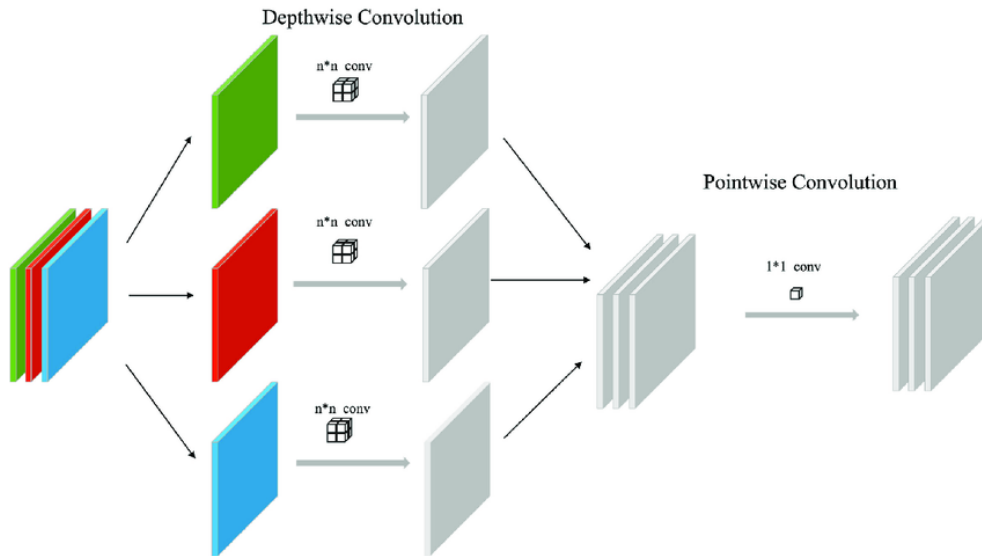


Figura 6: Representacion de Depthwise Separate Convolution.

Este metodo es presente en los dos modelos que no vamos a comentar tan a fondo que son el modelo MobileNets (arXiv:1704.04861v1) y el modelo Xception. Consideramos que puede llegar a ser mas complejo la explicación para modelos que no deberian de tener tanta cabida en el mundo de la medicina. Consideramos que se deberian de tener los recursos necesarios debido a que ese pequeño porcentaje que se pierde para compensar con una mayor eficiencia en el coste es algo interesante en el mundo de la empresa pero no en el de la salud.

Ambos modelos muestran en sus respectivos papers una mejora de casi 90 % a la hora del coste computacional (de flops) y de variables que optimizar con una pérdida de precisión menor del 1 %.

3.5. Métricas de evaluación

Siguiendo el paper original, utilizamos las siguientes métricas:

- Accuracy (Exactitud): Proporción de predicciones correctas sobre el total.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision (Precisión): Proporción de positivos predichos que son realmente positivos.

$$Precision = \frac{TP}{TP + FP}$$

- Recall (Sensibilidad): Capacidad del modelo para identificar correctamente los positivos.

$$Recall = \frac{TP}{TP + FN}$$

- F1-Score: Media armónica de Precision y Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.6. Estrategia de entrenamiento en Google Colab

Para manejar las interrupciones de Google Colab (límites de tiempo y recursos), implementamos:

1. ModelCheckpoint: Guarda el modelo automáticamente al final de cada época (o cuando mejora la métrica de validación).
2. EarlyStopping: Detiene el entrenamiento si la métrica de validación no mejora durante un número determinado de épocas (patience).
3. Guardado en Google Drive: Todos los checkpoints se guardan en Google Drive para no perder el progreso entre sesiones.
4. Reanudación del entrenamiento: El código permite cargar un modelo guardado previamente y continuar el entrenamiento desde donde se detuvo.

5. Guardado del historial: Se guarda el historial de entrenamiento (loss y métricas por época) en formato JSON para poder reconstruir las curvas de aprendizaje.

4. Estructura del código

El proyecto está organizado en scripts Python independientes, cada uno encargado de una fase del pipeline:

1. `redimensionar_imagenes.py`: Redimensiona las 25,000 imágenes del dataset LC25000 de 768×768 a 64×64 píxeles, utilizando interpolación LANCZOS para máxima calidad. Genera el directorio `lung_colon_image_set_64x64/`.
2. `extraer_lbp_features.py`: Aplica la transformación LBP ($P=8$, $R=1$, `method='uniform'`) sobre cada canal RGB de las imágenes redimensionadas. Genera el directorio `lung_colon_image_set_64x64_lbp/`.
3. `entrenar_cnn.py`: Construye, entrena y evalúa la CNN básica sobre las imágenes LBP. Incluye carga de datos, split train/val (80/20), entrenamiento con callbacks (ModelCheckpoint, EarlyStopping) y evaluación sobre el test set.
4. `entrenar_resnet.py`: Construye, entrena y evalúa el modelo ResNet50 con transfer learning sobre las imágenes LBP.
5. `entrenar_inceptionresnetv2.py`: Construye, entrena y evalúa el modelo InceptionResNetV2 con transfer learning sobre las imágenes LBP. Incluye la adaptación del input de 64×64 a 75×75 .
6. Cada script de entrenamiento genera un directorio de resultados (`resultados_cnn/`, `resultados_resnet/`, `resultados_inceptionresnetv2/`) con:
 - El mejor modelo guardado (checkpoint por `val_accuracy`).
 - El modelo final tras el entrenamiento.
 - El historial de entrenamiento en formato JSON.
 - Las métricas de evaluación en formato JSON.

5. Resultados

5.1. Resultados del paper original (LBP features, 768×768)

Cuadro 2: Resultados Finales del Entrenamiento

Modelo	Accuracy	Precision	Recall	F1-Score
CNN	89.50 %	91.64 %	91.19 %	91.39 %
ResNet	94.25 %	95.37 %	95.68 %	95.73 %
InceptionResNetV2	99.98 %	99.99 %	99.99 %	99.99 %

5.2. Nuestros resultados (LBP features, 64×64)

Cuadro 3: Comparativa General de Modelos

Modelo Tiempo	Accuracy	Precision	Recall	F1-Score	Épocas
CNN ~43 min	85.11 %	87.84 %	85.11 %	85.64 %	26
ResNet50 ~38 min	20.01 %	4.00 %	20.00 %	6.67 %	29
InceptionResNetV2 ~61 min	19.89 %	4.01 %	19.88 %	6.68 %	20

5.3. Comparación con el paper (diferencia en puntos porcentuales)

Cuadro 4: Comparativa de Variación en el Rendimiento (Δ)

Modelo	Δ Accuracy	Δ Precision	Δ Recall	Δ F1-Score
CNN	-4.39	-3.80	-6.08	-5.75
ResNet50	-74.24	-91.37	-75.68	-89.06
InceptionResNetV2	-80.09	-95.98	-80.11	-93.31

5.4. Detalle por clase — CNN (único modelo funcional)

Cuadro 5: Métricas de Rendimiento por Clase

Clase	Precision	Recall	F1-Score	Support
colon_aca	95.07 %	81.00 %	87.47 %	500
colon_n	96.58 %	96.00 %	96.29 %	500
lung_aca	63.31 %	89.40 %	74.13 %	500
lung_n	99.23 %	77.40 %	86.97 %	500
lung_scc	85.00 %	81.76 %	83.35 %	499

5.5. Matrices de confusión

CNN (85.11 % accuracy):

ResNet50 (20.01 % accuracy): Colapso completo: clasifica todas las muestras como colon_aca.

InceptionResNetV2 (19.89 % accuracy): Colapso completo: clasifica casi todas las muestras como colon_aca.

Cuadro 6: Matriz de Confusión

	colon_aca	colon_n	lung_aca	lung_n	lung_scc
colon_aca	405	16	59	0	20
colon_n	15	480	3	0	2
lung_aca	0	0	447	3	50
lung_n	0	1	112	387	0
lung_scc	6	0	85	0	408

5.6. Curvas de entrenamiento

CNN: El entrenamiento mostró una convergencia clara en el accuracy de train (de 74 % a 99.3 % en 26 épocas), aunque el accuracy de validación fue muy inestable, oscilando entre el 20 % y el 85 %. La val_loss mostró una variabilidad extrema (desde 0.65 hasta 45.9), indicativo de un severo sobreajuste. El early stopping detuvo el entrenamiento tras 26 épocas. A pesar de la inestabilidad en validación, el mejor checkpoint logró generalizar razonablemente al test set (85.11 %).

ResNet50: Tanto el accuracy de train como el de validación se mantuvieron estancados en 19.5-20 % durante las 29 épocas completas. El loss de train convergió a 1.61 (valor cercano a $\ln(1/5)$ aproximadamente 1.609, lo cual confirma predicciones uniformemente aleatorias o de una sola clase). El modelo nunca logró aprender patrones discriminativos.

InceptionResNetV2: Comportamiento análogo a ResNet50, con el accuracy estancado en 19-20 % durante 20 épocas. El loss de train convergió también a 1.61. El early stopping detuvo el entrenamiento ante la ausencia de mejora.

6. Conclusión

6.1. Resumen hallazgos

De los tres modelos evaluados, únicamente la CNN entrenada desde cero logró aprender a clasificar las imágenes histopatológicas del dataset LC25000 con features LBP a resolución 64×64 , alcanzando un accuracy del 85.11 %. Los modelos de transfer learning (ResNet50 e InceptionResNetV2) sufrieron un colapso total, prediciendo una sola clase para todas las muestras, con un accuracy equivalente al azar (aprox. 20 %).

6.2. Análisis del rendimiento de la CNN

La CNN básica obtuvo resultados razonables pese a la drástica reducción de resolución (de 768×768 a 64×64 , una reducción de 144x en el número de píxeles). La diferencia respecto al paper fue de solo 4.39 puntos porcentuales en accuracy (85.11 % vs. 89.50 %).

Analizando el rendimiento por clase:

- Las clases de colon (colon_aca y colon_n) obtuvieron los mejores resultados, con F1-scores de 87.47 % y 96.29 % respectivamente, lo que sugiere que las texturas LBP a baja resolución conservan suficiente información para distinguir tejido benigno de adenocarcinoma de colon.
- La clase lung_aca (adenocarcinoma de pulmón) presentó la precisión más baja (63.31 %), lo que indica que muchas muestras de otras clases fueron clasificadas erróneamente como lung_aca. La matriz de confusión confirma que 112 muestras de lung_n y 85 de lung_scc fueron clasificadas incorrectamente como lung_aca.
- La confusión entre las tres clases de pulmón (lung_aca, lung_n, lung_scc) sugiere que, a baja resolución, la información de textura LBP no es suficiente para distinguir de forma fiable los diferentes tipos de tejido pulmonar, que comparten patrones histológicos más similares entre sí que respecto al tejido de colon.

6.3. Análisis del fracaso de ResNet50 e InceptionResNetV2

El colapso de los modelos de transfer learning es el resultado más significativo y contraintuitivo de este trabajo. Mientras el paper original reporta un 94.25 % para ResNet y un 99.98 % para InceptionResNetV2 (ambos con LBP a 768×768), nuestros resultados muestran un fracaso absoluto. Las causas probables son:

1. Incompatibilidad entre los pesos preentrenados y la entrada LBP a baja resolución: Los pesos de ImageNet fueron aprendidos sobre imágenes naturales (objetos, animales, paisajes) a resolución alta (224×224 o superior). Las imágenes LBP a 64×64 representan un dominio radicalmente diferente: texturas binarias codificadas con muy pocos niveles de intensidad (0-9 valores posibles por píxel con LBP uniform, $P=8$). Los filtros convolucionales preentrenados, diseñados para detectar bordes, texturas y formas naturales, no son capaces de extraer características útiles de este tipo de representación tan diferente.
2. Resolución insuficiente para las arquitecturas profundas: ResNet50 tiene 50 capas con múltiples operaciones de reducción espacial (stride-2 convolutions, pooling). Con una entrada de 64×64 , el mapa de características se reduce rápidamente a dimensiones muy pequeñas (1×1 o 2×2) en las capas intermedias, eliminando toda la información espacial antes de que las capas más profundas puedan procesarla. InceptionResNetV2 tiene un problema aún mayor: su input mínimo es de 75×75 , por lo que fue necesario un upsampling de $64 \rightarrow 75$, que introduce interpolación artificial sin añadir información real.
3. Problema de la escala de gradientes (gradient flow): Al hacer fine-tuning completo (todas las capas entrenables) de modelos con millones de parámetros (aprox. 23.6M para ResNet50, aprox. 54.3M para InceptionResNetV2) sobre un dataset relativamente pequeño de imágenes LBP a baja resolución, se produce un desajuste masivo. Los gradientes que fluyen desde la capa de salida deben recorrer decenas o cientos de capas para actualizar los pesos preentrenados. Con una señal de entrada tan pobre (LBP 64×64), el modelo no consigue encontrar un mínimo útil y colapsa hacia una solución trivial: predecir siempre la clase más

frecuente (o la primera en el orden).

4. Colapso a la solución trivial: Las matrices de confusión de ambos modelos muestran que todas (o casi todas) las muestras se clasifican como `colon_aca`. Este patrón clásico de colapso ocurre cuando la función de loss se queda estancada en un mínimo local donde el modelo predice una distribución constante. El loss de ambos modelos convergió entorno a 1.61, que es exactamente $\ln(1/5)$ aprox. 1.609, es decir, la entropía cruzada de predecir uniformemente (o una sola clase) en un problema de 5 clases.

6.4. ¿Por qué la CNN sí funciona y los modelos preentrenados no?

La clave está en la diferencia entre aprender desde cero y adaptar conocimiento previo.

La CNN básica parte de pesos aleatorios y tiene una arquitectura mucho más simple (3 bloques convolucionales, aprox. 6.8M parámetros totales incluyendo las capas densas). Al no tener conocimiento previo que "desaprender", puede adaptarse directamente a las peculiaridades de las imágenes LBP a baja resolución. Sus filtros convolucionales aprenden desde cero a detectar los patrones de textura binaria específicos del dominio histopatológico.

Los modelos preentrenados llevan incorporado un sesgo fuerte hacia las características de ImageNet. Con imágenes LBP a 64×64 , la señal de entrada es tan diferente a lo que estos modelos "esperan" que el proceso de fine-tuning no logra superar el sesgo de los pesos preentrenados. En el paper original, con resolución 768×768 , las imágenes LBP aún contienen suficiente estructura espacial y riqueza de textura para que el fine-tuning funcione; a 64×64 , esa información se pierde.

6.5. Lecciones aprendidas

El transfer learning no es universalmente beneficioso: Cuando el dominio de origen (ImageNet: imágenes naturales) difiere drásticamente del dominio objetivo (imágenes LBP histopatológicas a baja resolución), los pesos preentrenados pueden ser contraproducentes.

La resolución es crítica para las features LBP: LBP captura patrones de textura local. Al reducir la resolución de 768×768 a 64×64 , la cantidad y calidad de los micro-patrones de textura se reduce enormemente. Sin embargo, la CNN básica demuestra que aún queda suficiente información para una clasificación razonable (85.11 %).

La complejidad del modelo debe ser proporcional a la complejidad de la entrada: Para entradas de baja resolución con información limitada, un modelo simple y entrenado desde cero puede superar ampliamente a modelos complejos preentrenados.

6.6. Posibles mejoras

Para mejorar los resultados de los modelos de transfer learning en este escenario, se podrían explorar:

- Congelar las capas del modelo base en lugar de hacer fine-tuning completo.
- Usar una resolución intermedia (por ejemplo, 128×128 o 224×224) que retenga más información.
- Aplicar una tasa de aprendizaje diferencial (más baja para las capas preentrenadas).
- Entrenar primero solo la cabeza de clasificación y luego descongelar progresivamente las capas del modelo base.
- Omitir la transformación LBP y usar las imágenes redimensionadas directamente, ya que los modelos preentrenados están diseñados para trabajar con imágenes naturales, no con representaciones de textura.

6.7. Conclusión final

Este trabajo demuestra que la replicación de resultados publicados es un ejercicio valioso que no siempre produce los resultados esperados. La reducción drástica de resolución (de 768×768 a 64×64) impacta de forma desigual a los distintos modelos: mientras la CNN básica mantiene un rendimiento aceptable (85.11 % vs. 89.50 % del paper), los modelos de transfer learning colapsan completamente. Esto subraya la importancia de considerar la compatibilidad entre el preprocesamiento aplicado, la resolución de las imágenes y la arquitectura del modelo al diseñar pipelines de clasificación de imágenes médicas.

Bibliografía

Borkowski, A. A., Bui, M. M., Thomas, L. B., Wilson, C. P., DeLand, L. A., & Mastorides, S. M. (2019). Lung and Colon Cancer Histopathological Image Dataset (LC25000).

Alsubai, S. (2024). Transfer learning based approach for lung and colon cancer detection using local binary pattern features and explainable artificial intelligence (AI) techniques. **PeerJ Computer Science**, 10, e1996.

Bray, F., Ferlay, J., Soerjomataram, I., Siegel, R. L., Torre, L. A., & Jemal, A. (2018). Global cancer statistics 2018. **CA: A Cancer Journal for Clinicians**, 68(6), 394–424.

Simonyan, K., Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

Tan, M., Le, Q. (2019, May). Efficientnet: Rethinking model scaling for convolutional neural networks. In International conference on machine learning (pp. 6105-6114). PMLR.

He, K., Zhang, X., Ren, S., Sun, J. (2015). Deep residual learning for image

recognition. arXiv preprint arXiv:1512.03385.

Szegedy, C., Ioffe, S., Vanhoucke, V., Alemi, A. A. (2017). "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning". Proceedings of the AAAI Conference on Artificial Intelligence.

David Solanas Sanz - Trabajo Fin de Grado "Predicción del diagnóstico de la enfermedad de Alzheimer mediante deep-learning en imágenes 18F-FDG PET". Universidad de Zaragoza.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... Adam, H. (2017). Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1251-1258).

Github

<https://github.com/alvaroinclan/proyecto-CNN>